House Price Prediction

Data Analysis using Python

By Priyanka Mathur

Project Overview

- House price prediction estimates property selling prices using factors like **physical conditions, concept, and location**. These factors help developers, agents, and buyers make better decisions.
- The project involves **feature engineering** to create a machine learning model that predicts house prices by finding relationships between features and sale price.

Dataset Overview

The dataset contains various features related to real estate properties for predicting house prices (SalePrice).

Property ID

Unique identifier for each house

Property Type

MSSubClass, MSZoning, BldgType classifications

Size & Area

LotArea, TotalBsmtSF measurements

Temporal Data

YearBuilt, YearRemodAdd construction years

Condition

OverallCond ratings on scale of 1-10

Target Variable

SalePrice property's sale price

Basic Data Exploration:-

Data loading:-

```
import pandas as pd
house_df = pd.read_excel(r'C:\Users\shavi\Downloads\priyanka project\Python\Project\HousePrediction.xlsx')
```

[2]

Important necessaries and essential libraries:-

- Pandas
- Numpy
- Matplotlib
- Seaborn
- Sklearn

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

Basic Data Exploration:-

Display all the columns of DataFrame

```python
house_df.columns
```

```
Index(['Id', 'MSSubClass', 'MSZoning', 'LotArea', 'LotConfig', 'BldgType',
       'OverallCond', 'YearBuilt', 'YearRemodAdd', 'Exterior1st', 'BsmtFinSF2',
       'TotalBsmtSF', 'SalePrice'],
      dtype='object')
```

Read the data and display the first 100 rows

```python
house_df.head(100)
```

Python

Id	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt	YearRemodAdd	Exterior1st	BsmtFinSF2	TotalBsmtSF	SalePrice
0	60	RL	8450	Inside	1Fam	5	2003	2003	VinylSd	0.0	856.0	208500.0
1	20	RL	9600	FR2	1Fam	8	1976	1976	MetalSd	0.0	1262.0	181500.0
2	60	RL	11250	Inside	1Fam	5	2001	2002	VinylSd	0.0	920.0	223500.0
3	70	RL	9550	Corner	1Fam	5	1915	1970	Wd Sdng	0.0	756.0	140000.0
4	60	RL	14260	FR2	1Fam	5	2000	2000	VinylSd	0.0	1145.0	250000.0
...	...	...	...	...	...	...	...	...	...	...	...	...
95	60	RL	9765	Corner	1Fam	8	1993	1993	VinylSd	0.0	680.0	185000.0
96	20	RL	10264	Inside	1Fam	5	1999	1999	VinylSd	0.0	1588.0	214000.0
97	20	RL	10921	Inside	1Fam	5	1965	1965	HdBoard	0.0	960.0	94750.0
98	30	RL	10625	Corner	1Fam	5	1920	1950	Wd Sdng	0.0	458.0	83000.0
99	20	RL	9320	Inside	1Fam	5	1959	1959	Plywood	0.0	950.0	128950.0

Give column insights

```python
house_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2919 entries, 0 to 2918
Data columns (total 13 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Id            2919 non-null   int64
 1   MSSubClass    2919 non-null   int64
 2   MSZoning      2915 non-null   object
 3   LotArea       2919 non-null   int64
 4   LotConfig     2919 non-null   object
 5   BldgType      2919 non-null   object
 6   OverallCond   2919 non-null   int64
 7   YearBuilt     2919 non-null   int64
 8   YearRemodAdd  2919 non-null   int64
 9   Exterior1st   2918 non-null   object
 10  BsmtFinSF2    2918 non-null   float64
 11  TotalBsmtSF   2918 non-null   float64
 12  SalePrice     1460 non-null   float64
dtypes: float64(3), int64(6), object(4)
memory usage: 296.6+ KB
```

Data Cleaning & Analysis

- Checking for Missing Values

Data Cleaning and analysis – Task 2

```
[11]
house_df.isnull().sum()
```

```
Id                0
MSSubClass        0
MSZoning          4
LotArea           0
LotConfig         0
BldgType          0
OverallCond       0
YearBuilt         0
YearRemodAdd      0
Exterior1st       1
BsmtFinSF2        1
TotalBsmtSF       1
SalePrice      1459
dtype: int64
```

- Calculate Mean Sales Price for Missing/Present Information

```
[4]
# filling the null values of sale price with mean
house_df['SalePrice'] = house_df['SalePrice'].fillna(house_df['SalePrice'].mean())
```

```
[4]
house_df.isnull().sum()
```

```
Id              0
MSSubClass      0
```

- Count of Numerical Features

```
# Select numeric columns
numeric_columns = house_df.select_dtypes(include=['number'])
# Count them
count = numeric_columns.shape[1]
print("Number of numerical columns:", count)
```

✓ 0.0s

```
Number of numerical columns: 9
```

- Print the First Five Rows of Numerical Values

```
house_df.sort_values(by=numeric_columns.columns.tolist(),ascending=False).head(5)
```

✓ 0.0s

	Id	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt
2918	2918	60	RL	9627	Inside	1Fam	5	1993
2917	2917	85	RL	10441	Inside	1Fam	5	1992
2916	2916	20	RL	20000	Inside	1Fam	7	1960
2915	2915	160	RM	1894	Inside	TwnhsE	5	1970
2914	2914	160	RM	1936	Inside	Twnhs	7	1970

- Compare the Difference Between Year Features and SalePrice

```
#Q6) Compare the Difference Between Year Features and SalePrice
year_features = ['YearBuilt', 'YearRemodAdd', 'GarageYrBlt', 'YrSold']
for feature in year_features:
    if feature in house_df.columns:
        print(f"Comparison of {feature} and SalePrice:")
        print(house_df[[feature, 'SalePrice']].corr())
        print()
```

Data Cleaning & Analysis

- Relationship Between Discrete Variables and Sales Price

```
#Q7) Relationship Between Discrete Variables and Sales Price
for col in Discrete_columns.columns:
    if col!='SalePrice':
        print("Relationship between {col} and SalePrice:")
        print(house_df[[col, 'SalePrice']].corr())
        print()
```

✓ 0.0s

- Relationship Between Continuous Variables and Sales Price

```
if 'TotalBsmtSF' in house_df.columns and 'SalePrice' in house_df.columns:
    print("Relationship between TotalBsmtSF and SalePrice:")
    print(house_df[['TotalBsmtSF', 'SalePrice']].corr())
```

✓ 0.0s

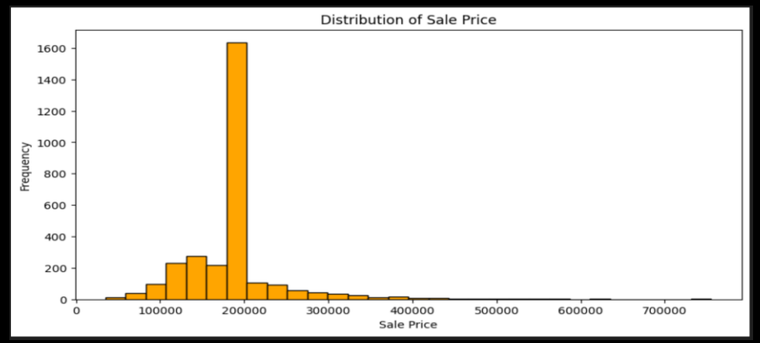
```
Relationship between TotalBsmtSF and SalePrice:
             TotalBsmtSF  SalePrice
TotalBsmtSF     1.000000   0.431912
SalePrice       0.431912   1.000000
```

- Histogram Analysis for Continuous Variables

```
# Histogram
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.hist(house_df['SalePrice'], bins=30, color='orange', edgecolor='black')
plt.title('Distribution of Sale Price')
plt.xlabel('Sale Price')
plt.ylabel('Frequency')
plt.show()
```

✓ 1.3s



- Logarithmic Transformation

```
# Logarithmic Transformation
import numpy as np
house_df['Log_SalePrice'] = np.log(house_df['SalePrice'])
```

✓ 0.0s

Feature Relationships

Year Features

Compare YearBuilt and YearRemodAdd with sale price

```
#Q6) Compare the Difference Between Year Features and SalePrice
year_features = ['YearBuilt', 'YearRemodAdd', 'GarageYrBlt', 'YrSold']
for feature in year_features:
    if feature in house_df.columns:
        print(f"Comparison of {feature} and SalePrice:")
        print(house_df[[feature, 'SalePrice']].corr())
        print()
```

[12]

```
Comparison of YearBuilt and SalePrice:
           YearBuilt  SalePrice
YearBuilt   1.000000   0.369442
SalePrice   0.369442   1.000000

Comparison of YearRemodAdd and SalePrice:
               YearRemodAdd  SalePrice
YearRemodAdd       1.000000   0.355167
SalePrice          0.355167   1.000000
```

Discrete Variables

Analyze OverallCond influence using statistics

```
#Q7) Relationship Between Discrete Variables and Sales Price
for col in Discrete_columns.columns:
    if col!='SalePrice':
        print("Relationship between {col} and SalePrice:")
        print(house_df[[col, 'SalePrice']].corr())
        print()
```

Continuous Variables

Check LotArea and TotalBsmtSF correlation

```
if 'TotalBsmtSF' in house_df.columns and 'SalePrice' in house_df.columns:
    print("Relationship between TotalBsmtSF and SalePrice:")
    print(house_df[['TotalBsmtSF', 'SalePrice']].corr())
```

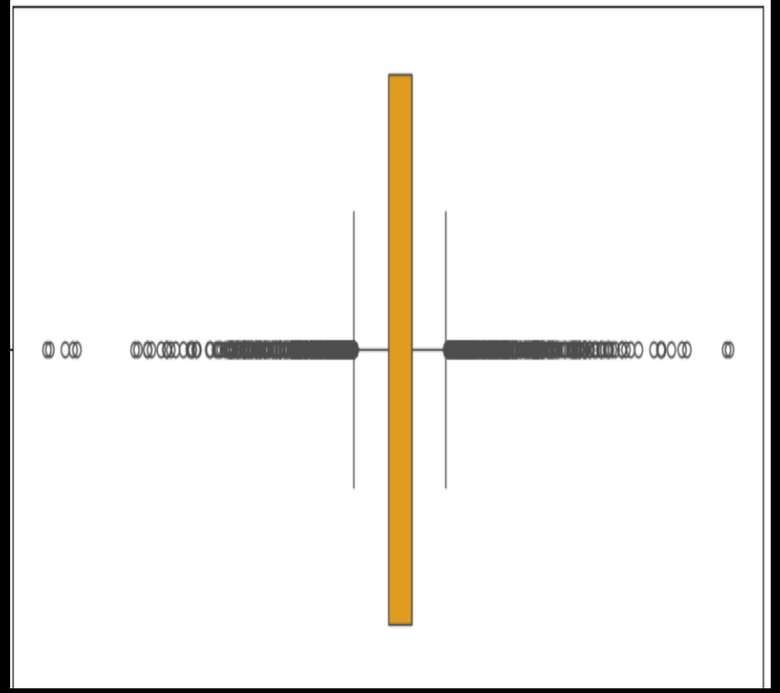
Feature Engineering

1 Find Outliers

```
# Box Plot

import seaborn as sns
plt.figure(figsize=(10,5))
sns.boxplot(x=house_df['Log_SalePrice'], color='orange')
```

```
es: xlabel='Log_SalePrice'>
```



2 Categorical Features

```
#  Relationship Between Categorical Features and SalePrice
Categorical_features = house_df.select_dtypes(include=['object']).columns
for feature in Categorical_features:
    if feature in house_df.columns:
        print(f"Relationship between {feature} and SalePrice:")
        print(house_df[[feature,'SalePrice']].groupby(feature).mean())
```

3 Correlation Analysis

```
for col in numeric_columns:
    if col != 'SalePrice':
        print(f"Relationship between {col} and SalePrice")
        print(house_df[[col, 'SalePrice']].corr())
```

4 Filling Missing Values

```
#filling missing or null values with mean and mode
for col in house_df.columns:
    if house_df[col].dtype == 'object':
        m= house_df[col].mode()[0]
        house_df[col] = house_df[col].fillna(m)
    else:
        me = house_df[col].mean()
        house_df[col] = house_df[col].fillna(me)
```

Advanced Questions

1 Location Influence

How does zoning and lot configuration affect house price?

```
if 'LotArea'in house_df.columns and 'SalePrice' in house_df.columns:
    print(f"comparison of Lot Area and SalePrice:")
    print(house_df[['LotArea','SalePrice']].corr())
```

```
comparison of Lot Area and SalePrice:
              LotArea  SalePrice
LotArea      1.000000   0.236105
SalePrice    0.236105   1.000000
```

2 Condition Impact

What is the impact of overall house condition on sale price?

```
if 'OverallCond' in house_df.columns and 'SalePrice' in house_df.column
    print(f"Comparison of OverallCond and SalePrice")
    print(house_df[['OverallCond','SalePrice']].corr())
✓ 0.0s
```

```
Comparison of OverallCond and SalePrice
             OverallCond  SalePrice
OverallCond     1.000000  -0.055036
SalePrice      -0.055036   1.000000
```

3 Basement Effect

Does TotalBsmtSF significantly affect house price?

```
if 'TotalBsmtSF' in house_df.columns and 'SalePrice' in house_df.columns:
    print(f"Comparison of TotalBsmtSF and SalePrice")
    print(house_df[['TotalBsmtSF' and 'SalePrice']].corr())
✓ 0.0s
```

```
Comparison of TotalBsmtSF and SalePrice
           SalePrice
SalePrice        1.0
```

Advanced Questions

4 Renovation Value

How do YearRemodAdd renovations influence property value?

```
if 'YearRemodAdd' in house_df.columns and 'SalePrice' in house_df.columns:
    print("Comparison of YearRemodAdd and SalePrice")
    print(house_df[['YearRemodAdd' and 'SalePrice']].corr())
```

✓ 0.0s

```
Comparison of YearRemodAdd and SalePrice
             SalePrice
SalePrice          1.0
```

Generate + Code + Markd

5 Location Influence

How does zoning affect house price?

```
if 'MSZoning' in house_df.columns and 'SalePrice' in house_df.columns:
    print(f"Comparison of MSZoning and SalePrice:")
    print(house_df[['MSZoning'and 'SalePrice']].corr())
```

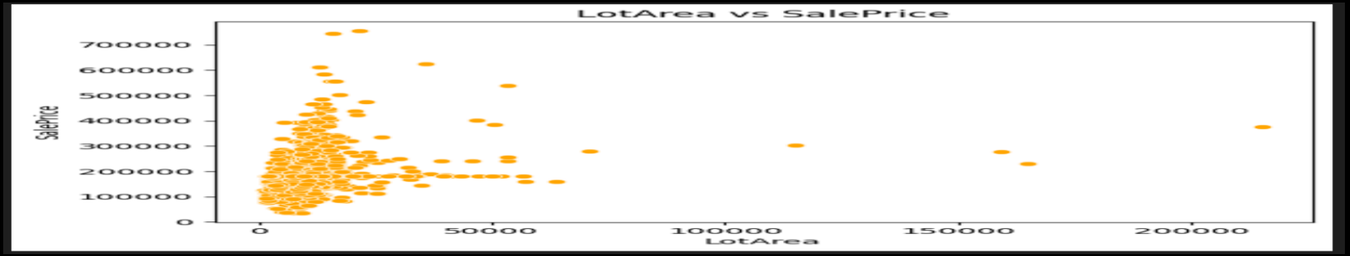
✓ 0.0s

```
Comparison of MSZoning and SalePrice:
             SalePrice
SalePrice          1.0
```

Visualizations

- Lot Area vs Sales Price

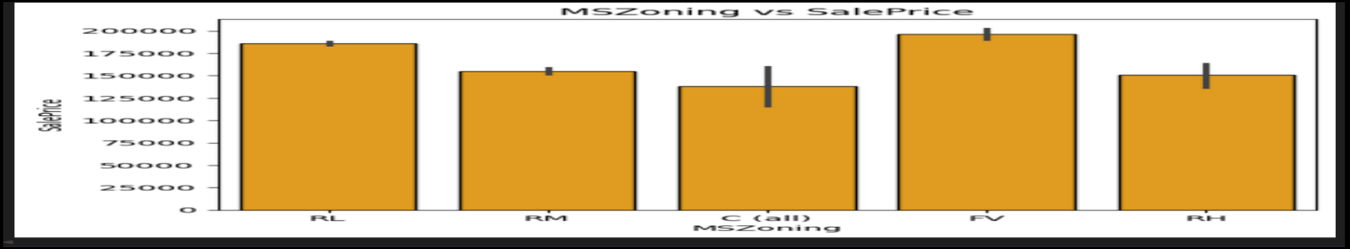
```
import matplotlib.pyplot as plt
import seaborn as sns
sns.scatterplot(x = 'LotArea',y = 'SalePrice', data =house_df,color = 'orange' )
plt.title('LotArea vs SalePrice')
plt.show()
```



- **Insights:**
  Larger area → generally higher price
  Some outliers present

- MS zoning vs Sales Price

```
#Bar chart
plt.Figure(figsize=(10,5))
sns.barplot(x='MSZoning',y='SalePrice',data = house_df, color = 'orange', edgecolor = 'Black' )
plt.xlabel='MSZoning'
plt.ylabel='SalePrice'
plt.title("MSZoning vs SalePrice")
plt.show()
```

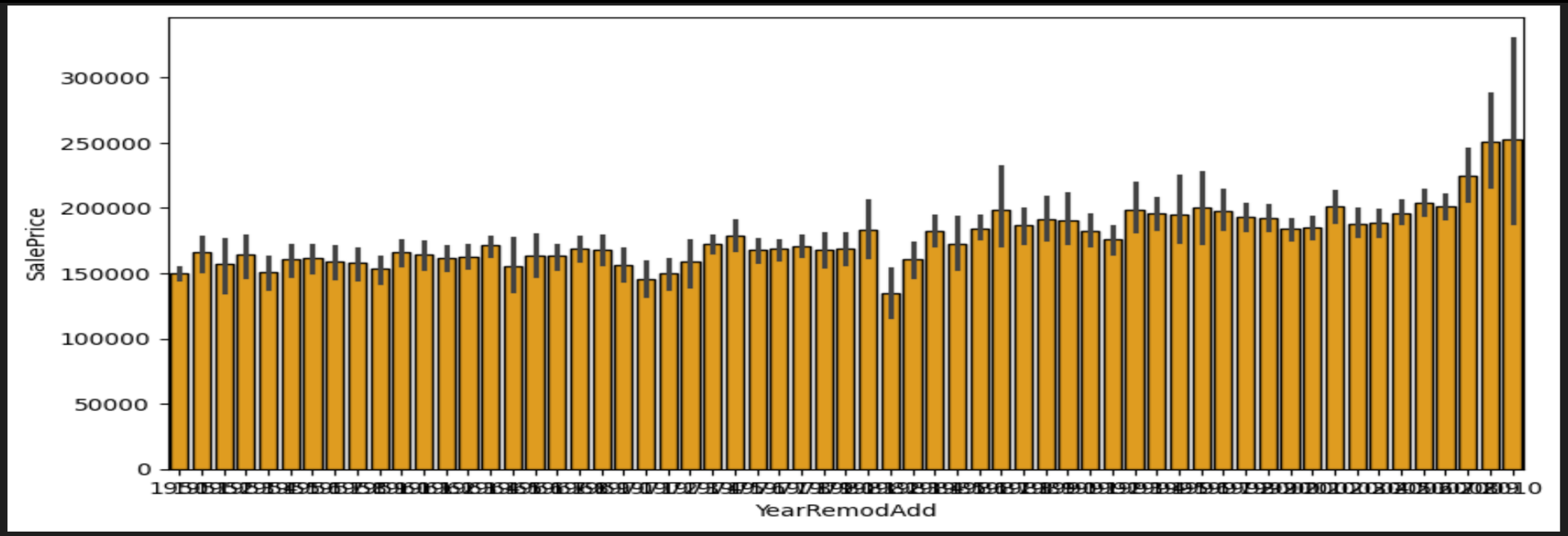


- **Insights**

FV (Floating Village Residential) has the **highest average sale price (~₹195K–200K)**
•ndicates **premium or high-value residential area**

Visualizations

- YearRemoAdd vs Saleprice



- Insights

•As **YearRemodAdd increases**, **SalePrice also increases**
•Recently renovated houses have **higher value**

Key Takeaways

Comprehensive Analysis

Complete data exploration, cleaning, and feature engineering process

Multiple Features

Analysis of categorical, numerical, and temporal variables

Practical Application

Real-world insights for developers, agents, and buyers

Conclusions

•The project successfully analyzed various factors affecting house prices using Python and data visualization techniques.

•Key features such as **location (MSZoning), overall condition, basement area, and renovation year** were found to have a significant impact on sale prices.

•Data preprocessing steps like **handling missing values, outlier detection, and feature engineering** improved the quality of the dataset.

•Exploratory Data Analysis (EDA) helped in identifying **important patterns and relationships** between features and the target variable.

•The machine learning model was able to **predict house prices with reasonable accuracy**, demonstrating the effectiveness of the selected features.

•Visualizations clearly showed that:

- Better location and condition → higher prices
- Larger area and basement → higher prices
- Recent renovations → premium pricing